# EchoPin: Implementation Phase

**Project:** EchoPin, privacy-first group presence over Bluetooth and Wi-Fi, with an optional central server
**Version documented:** 0.5.0 (app), 1.1.0 (server) **Repository:** https://github.com/BEEKONTek/EchoPin
**Author:** Botlhale Pono Maruma **Date:** 12 November 2026

This document covers the implementation phase: how the system was coded, how it was tested, the system test cases with their results, and how it is installed. It refers to the live repository for every claim, so each statement can be checked.

---

## 5.1 Introduction

EchoPin keeps a small group together without tracking anyone. Phones in a **session** see each other as distances on a pinwheel, raise alerts when someone crosses an agreed boundary, check in on each other, and send an SOS that every member receives, all phone to phone over Bluetooth Low Energy and Wi-Fi. No coordinate is ever measured, stored or sent; distance comes from signal strength on the receiving phone and is deliberately jittered before it reaches the screen.

The implementation phase delivered:

1. **The app**, for Android, iOS, Linux, macOS, Windows and the web (demo only), built with Flutter 3.44 and Dart 3.12.
2. **A shared protocol package** (`packages/echopin_protocol`) holding the sealed-box cryptography, device proofs, wire enumerations and the input limits used by both app and server.
3. **The optional server** (`server/`), Dart on PostgreSQL, adding verified accounts, device-bound sign-in, sealed emergency records that only an authorised responder can open, reports and diagnostics from every device, a staff console, an append-only audit chain and encrypted backups. A live instance runs on free hosting (Render, Supabase, Brevo).
4. **Tooling**: continuous integration for lint, tests and builds on every push, a one-command server test suite, a one-command system test, an installer that works by QR code, and a web demo published to GitHub Pages.

The guiding constraints throughout were: the app must stay fully usable with no server; no location data may exist anywhere in the system; every failure must be explained to the person in plain words; and the server must never trust the app.

Related documents: `README.md` (overview), `docs/ARCHITECTURE.md` (diagrams), `docs/CODE_MAP.md` (file-by-file guide), `docs/PROTOCOL.md` (wire format), `docs/THREAT_MODEL.md` and `docs/SECURITY_REVIEW.md` (security), `docs/SERVER.md` and `docs/DEPLOY_HOSTED.md` (operations).

---

## 5.2 Coding

### 5.2.1 Languages, frameworks and tools

| Part | Technology | Why |
|---|---|---|
| App | Flutter 3.44 / Dart 3.12 | one code base for six platforms; all cryptography in Dart so it is identical everywhere and testable on a desktop |
| Native plugins | Kotlin (Android), Swift (iOS) | Bluetooth LE advertising, scanning and GATT; they move opaque bytes only |
| Shared package | Dart (`package:cryptography`) | X25519 + HKDF + XChaCha20-Poly1305 sealed boxes, Ed25519 device proofs |
| Server | Dart (`shelf`, `postgres`) | same language as the app, one team, shared models |
| Database | PostgreSQL 16–18 | row-level security, triggers for an append-only audit log |
| Email | Brevo HTTP API (SMTP and Resend supported) | free tier; Render's free plan blocks SMTP |
| Hosting | Render (Docker), Supabase (PostgreSQL + Storage) | free, HTTPS, daily encrypted backups |
| CI | GitHub Actions | lint, tests, builds for every platform, Pages deployment |

### 5.2.2 Code structure

```
lib/                     the app
  core/                  EchoError: every failure as a typed, human sentence
  security/              invite code → keys (HKDF), Ed25519 identity, envelopes,
                         SecureMessenger (decrypt, verify, pin, replay, rate limit)
  session/               SessionEngine (membership, pings, SOS, receipts, retries)
  transport/             Wi-Fi (Bonjour + TCP), in-memory loopback
  services/              Bluetooth transport + bridge, MeshService, noise, alerts,
                         notifications, connectivity, history, settings, PIN auth
  providers/             ChangeNotifiers: Session, Alert, Settings, Admin,
                         Account, AccountLock, Outbox
  account/               server client, AccountProvider, Outbox, SessionRecorder
  screens/ widgets/ ui/  home, session, pinwheel, trays, admin, account, console
android/ ios/            native BLE plugins
packages/echopin_protocol sealed boxes, device proofs, enums, ProtocolLimits
server/
  lib/src/api/           http.dart (parsing, validation, error mapping), routes.dart
  lib/src/auth/          AuthService, passwords (Argon2id), TOTP, breach check
  lib/src/               AdminService, TelemetryService, RetentionService
  lib/src/security/      keys, audit chain, custodian keys, rate limits
  lib/src/backup/        encrypted backups, storage backends
  lib/src/db/            pool and migrations (7 so far)
  bin/                   server.dart, echopin_admin.dart (operator CLI)
  tool/                  test.sh, live_check.sh + live_check.dart (system test)
scripts/                 share_apk.sh (install by QR code)
```

### 5.2.3 Coding standards

- **Analyzer and formatter are law.** `flutter analyze` must report no issues across all three packages and `dart format` must produce no change; CI fails otherwise (`.github/workflows/lint.yml`).
- **Lints**: `flutter_lints` plus the repository's `analysis_options.yaml` (typed locals, braces on every control statement, no print statements in library code).
- **Errors carry a sentence.** In the app every failure is an `EchoError` subclass with a human message and, where possible, a one-tap fix. On the server every expected failure is an `ApiException(status, code, message)`; clients switch on the code and show the message. Stack traces and database wording never reach a client.
- **Comments explain why.** Each security-relevant block starts with a short note of the threat it addresses (`SECURITY:` tags in auth code).
- **Shared constants, not copies.** Text limits live once in `ProtocolLimits` and are read by the app's fields and the server's checks.
- **Version control.** Conventional commit messages (`feat:`, `fix:`, `docs:`); `main` is the release branch and `update` the working branch; every push runs the full CI.

### 5.2.4 Key modules

| Module | Responsibility | Notable decisions |
|---|---|---|
| `security/session_credentials.dart` | invite code ↔ session id and key | 16 base-32 characters (~78 bits) plus a 2-character checksum that works offline on any device; the checksum catches typos before any crypto runs |
| `security/secure_messenger.dart` | envelope decrypt/verify, sender pinning, replay window, rate limits | sender ids are derived from public keys and pinned on first sight, so an impersonator is detected at the second message |
| `session/session_engine.dart` | hello / ping / receipt / reply / bye, retries, SOS lifecycle | pings retry until acknowledged; SOS state is replayed to late joiners and resolved everywhere on *I'm safe* |
| `services/location_engine.dart` | privacy noise | jitter is fixed per peer per minute (FNV-1a seeded), so repeated samples cannot be averaged away |
| `widgets/pinwheel_painter.dart` | the radar | 100 m scale; peers without a measured bearing are spread evenly and the caption says *direction unknown*, so no direction is ever invented |
| `account/account_provider.dart` | sign-up, sign-in, tokens, device key, two-step, custodian keys | offline-first: a saved sign-in survives restarts; local pre-checks on email, name, password and codes before any request |
| `providers/account_lock_provider.dart` | PIN in front of the signed-in account | PIN per sign-in, 5-minute idle lock on the wall clock, third lockout signs out |

| Module | Responsibility | Notable decisions |
| --- | --- | --- |
| `account/outbox.dart`, `session_recorder.dart` | sealed emergency records | sealed to the custodian key, then the escrow key; checkpoint uploaded within seconds of an SOS; queued offline |
| `server/lib/src/auth/auth_service.dart` | accounts | Argon2id (19 MiB, t=2), uniform response time, per-IP/email/device limits in the database, known-device bypass so a stranger cannot lock the owner out, 8-digit codes with a daily cap |
| `server/lib/src/admin_service.dart` | console and break-glass | roles re-read on every request; unseal writes the grant and audit row before decrypting and emails every admin |
| `server/lib/src/security/audit.dart` | audit chain | HMAC-SHA256 chain keyed from a secret outside the database; UPDATE/DELETE/TRUNCATE refused by triggers; chain tip anchored with every backup |
| `server/lib/src/backup/` | backups | `pg_dump` → AES-256-GCM 1 MiB frames → private Supabase bucket in 40 MB parts; verify and restore from the CLI |

### 5.2.5 Input validation and error handling

Every piece of text a person or a device can send is bounded twice: in the app, so the limit is visible while typing, and on the server, which never trusts a client. The limits are one set of constants (`ProtocolLimits`) used on both sides.

| Input | Where it is entered | App check | Server check |
| --- | --- | --- | --- |
| Your name (session) | Home | max 24 characters, control/bidi characters stripped from network copies | n/a (phone to phone); peers strip and cap received names |
| Session name | Home | max 40 | n/a |
| Invite code | Home, QR, link | checksum, alphabet (no `0`, `1`), length; precise message per mistake | n/a |
| Account name | Sign-up | max 40, not empty | `cleanDisplayName`: control characters removed, max 40 |
| Email | Sign-in, sign-up, reset | max 254, shape `x@y.z` | `normalizeEmail`: RFC-shaped regex, lower-cased, max 254 |
| Password | Sign-in, sign-up, change, reset | max 256; sign-up: at least 10, not your email, not a common pattern | same policy, plus a breached-password lookup (HIBP k-anonymity, fails open) |
| Email code | Verify, reset | exactly 8 digits | constant-time compare, attempt cap, expiry, one generic error |

| Input | Where it is entered | App check | Server check |
|---|---|---|---|
| Authenticator code | Sign-in, two-step setup | exactly 6 digits | RFC 6238 window, replay refused |
| Account / admin PIN | Lock screens | 4–12 digits | local only; PBKDF2, lockout, lockout count |
| Report subject / body | Report a problem | max 120 / 4000 | same (`ProtocolLimits`) |
| Unseal reason / case reference | Console | max 1000 / 100 | same |
| Report note / resolution | Console | max 4000 / 2000 | same |
| Search text | Console | max 200 | 400 if longer |
| Custodian key | Console | prefix `ECHOPIN-CUSTODIAN-1:`, base64url seed of 32 bytes | n/a (never sent to the server) |
| Ids in URLs | app → server | n/a | `uuidParam` / `idParam`: anything that cannot be an id answers 404 before any query |
| Paging (`limit`, `offset`, `before`) | app → server | n/a | clamped to 1–500 / ≥ 0 |
| Request body | app → server | n/a | ≤ 64 KB, must be a JSON object; typed field accessors give precise 400s |
| Sealed uploads | app → server | sealed on the phone | ≤ 512 KB, key id must match a known escrow key, custodian seal required when configured, idempotent by id |
| Server settings | operator | n/a | refused at start-up with a message naming the variable (bad key bundle, dashboard URL pasted as API URL, CORS `*` outside development…) |

Error handling follows one rule on each side:

- **App.** Network and server failures become `OfflineError` or `AccountError(code, message)`; screens show `errorText(e)` and keep the person's input. Background work (`unawaited`) always catches and either audits or logs; the twelve deliberately silent catches in the code base are all shutdown or best-effort paths (closing radios, deleting a key that may not exist) and are commented as such.
- **Server.** One middleware (`errorsAndLogging`) converts every exception: `ApiException` → its status and code; a PostgreSQL *data* error (SQLSTATE class 22) → 400 without the database's wording; a uniqueness clash → 409; anything else → 500 with a generic message and a server-side log line (never the request body). Failures in mail and backups are recorded in the audit log and the backups table, so the console shows them without access to host logs.

---

## 5.3 Testing

### 5.3.1 Strategy

Testing runs at six levels, from pure functions to the live deployment. Everything below unit level uses real components: real cryptography, a real in-memory radio, a real PostgreSQL, a real HTTP server, the real Linux build of the app.

| Level | What runs | Where | Count |
|---|---|---|---|
| Unit | codes, key derivation, envelopes, PBKDF2 vectors, noise, models, settings migration | `test/security`, `test/models`, `test/services` | 103 |
| Component | SessionEngine with lossy and partitioned links, providers over a shared radio, account provider against a fake server, outbox and recorder | `test/session`, `test/providers`, `test/account` | 89 |
| Widget (end-to-end UI) | home, demo with answers, hold-to-send SOS, admin and account locks, sign-in and two-step, report filing, console, input limits | `test/widgets` | 32 |
| Protocol package | sealed boxes, device proofs | `packages/echopin_protocol/test` | 6 |
| Server | every endpoint against a throwaway PostgreSQL: registration, codes, sign-in, tokens, two-step, roles, uploads, break-glass, audit chain, backups and restore, rate limits, hardening regressions, input validation | `server/test` | 75 |
| Integration | real Bonjour discovery and TCP between two engines; the real app driven through sign-up, two-step, console and break-glass against a real server | `integration_test/` | 3 |
| System | 31 checks over HTTP against a freshly started server | `server/tool/live_check.sh` | 31 |
| Live deployment | health, database, email delivery, backups, data-API lock-down on the hosted instance | manual, recorded in §5.4 | 9 |

Totals at the time of writing: **239 app tests pass (1 skipped without a server), 75 server tests pass, 6 protocol tests pass, 31 of 31 system checks pass.**

### 5.3.2 Tools

- `flutter test` with `--coverage` in CI; widget tests use `FakeAsync` and a fake HTTP server (`test/helpers/fake_echopin_server.dart`) that implements the API faithfully, including device proofs, so sign-in flows are exercised end to end without a network.
- `server/tool/test.sh` starts a throwaway PostgreSQL cluster (`initdb` / `pg_ctl`), runs all server tests and tears it down; CI does the same with a PostgreSQL 16 service.
- `server/tool/live_check.sh` starts a real server with fresh keys and a custodian key, creates an admin, and runs `live_check.dart`, which acts as several phones and a console over plain HTTP.

- `integration_test/live_server_test.dart` drives the real Linux build of the app under Xvfb against that server, with screenshots at each step.
- GitHub Actions runs lint, all test levels except the system and live ones, and release builds for every platform on each push, publishing failing test names as annotations.

### 5.3.3 How a bug found in testing was handled

Each defect found during testing was fixed with a regression test in the same commit. Examples from this phase, all reproduced before fixing:

| Found by | Defect | Fix and test |
|---|---|---|
| system test | a member-proof tie (one proven, one not) proved both | strict majority of distinct accounts; tie → nobody proven; `api_test` "hardening" |
| system test | SOS counted once per upload, inflating the chart | `max()` per session, `count(distinct)` for the chart |
| GUI integration test | two-step dialog crashed inside a layout builder | fixed-size QR box |
| live deployment | server image had no CA certificates: mail and backups failed | `ca-certificates` in the image |
| live deployment | Supabase's public API could read and write every table | RLS enabled and grants revoked on every start; regression test creates the roles and proves access is gone |
| live deployment | `MAIL_FROM` as `Name <address>` refused by Brevo | parsed; provider's reason recorded in the audit log |
| live deployment | an interrupted backup blocked all later backups | cleanup no longer depends on the store being reachable; store errors recorded on the backup row |
| live deployment | dashboard URL pasted as `SUPABASE_URL` | normalised; refused with guidance when not a URL |
| CI | restore tests failed on the runner: `pg_restore` newer than the server | client pinned to the service's major |
| CI | one timing-sensitive widget test failed only on slow runners | tests wait for the fake server to be idle instead of a fixed delay |
| CI | the hosted-backup test built its own config and ignored the pinned client, so a PostgreSQL 18 dump went to a 16 server | all test configs share `pgDumpBinary` / `pgRestoreBinary`; reproduced locally against a PostgreSQL 16 container |
| CI | every change of a radio's reachable count re-broadcast hello, so five people joining at once sent each other up to six hellos | announce only when a radio comes back up; `session_engine_test` counts the frames a newcomer costs |
| CI | one check-in answer out of four was lost on fast runners: a group joining in the same instant sends each newcomer a direct hello, every phone overhears them, and the per-sender token bucket was charged for frames not addressed to it | the messenger charges the bucket only for frames addressed to the receiver; regression test in `secure_messenger_test` |

| Found by | Defect | Fix and test |
|---|---|---|
| review | malformed ids in URLs reached the database and produced 500s | `uuidParam` / `idParam`; `api_test` "input validation" |

---

## 5.4 System testing

### 5.4.1 Test environment

- **System under test**: server 1.1.0 built from `main`, PostgreSQL 18 locally (16 in CI, 17 hosted), app 0.5.0 Linux release build.
- **Local**: `server/tool/live_check.sh` (fresh database and keys each run) and the GUI integration test under Xvfb.
- **Hosted**: https://echopin-server-z3tg.onrender.com with Supabase (eu-west-1) and Brevo, checked from the public internet.
- **Phones**: Android phones with Bluetooth LE, app 0.5.0 release APK installed by QR code, tested outdoors and indoors on 13 November 2026.

### 5.4.2 Test cases and results

#### A. End-to-end over HTTP (`live_check.sh`, 31 checks, automated)

| ID | Area | Case | Expected | Result |
|---|---|---|---|---|
| TC-01 | Registration | password found in a public breach | refused with `breached_password` | Pass |
| TC-02 | Registration | valid sign-up | 202 `verification_sent` | Pass |
| TC-03 | Registration | timing of sign-up | ≥ 600 ms (uniform) | Pass |
| TC-04 | Enumeration | sign-up with an address already taken | identical 202 and timing | Pass |
| TC-05 | Email codes | code in the email | 8 digits | Pass |
| TC-06 | Pre-hijack | right code, different password | `password_mismatch` | Pass |
| TC-07 | Email codes | wrong code | one generic error, no attempt count | Pass |
| TC-08 | Enumeration | wrong code for an unknown address | identical answer | Pass |
| TC-09 | Verification | fresh code + sign-up password | account verified | Pass |
| TC-10 | Sign-in | device-proof sign-in | 200 with tokens | Pass |

| ID | Area | Case | Expected | Result |
|---|---|---|---|---|
| TC-11 | Lockout | 5 wrong passwords from a new device | 423 locked | Pass |
| TC-12 | Lockout | owner's known device during the lock | still signs in | Pass |
| TC-13 | Enumeration | 5 wrong passwords for an address with no account | locks identically | Pass |
| TC-14 | Staff two-step | admin without two-step opens the console | `two_step_required` | Pass |
| TC-15 | Two-step | enable with a wrong password | refused | Pass |
| TC-16 | Two-step | enable with password + code | enabled | Pass |
| TC-17 | Staff two-step | console after enabling | answers | Pass |
| TC-18 | Sealed records | double-sealed upload | accepted | Pass |
| TC-19 | Registration | a second person registers | 202 | Pass |
| TC-20 | Sign-in | second person signs in | 200 | Pass |
| TC-21 | Sealed records | single-sealed upload when a custodian is required | `custodian_required` | Pass |
| TC-22 | Membership proof | upload from a non-member | session flagged `unproven` | Pass |
| TC-23 | Membership proof | one member, one non-member | tie: nobody proven, both flagged | Pass |
| TC-24 | Roles | responder views the session | names, no email addresses | Pass |
| TC-25 | Break-glass | responder opens a record with a reason | only the inner seal returned | Pass |
| TC-26 | Custodian | custodian key on the device | record readable | Pass |
| TC-27 | Alerting | after the opening | every admin emailed | Pass |
| TC-28 | Audit | chain verification | keyed HMAC chain and backup anchor verify | Pass |
| TC-29 | Audit | session views and unseals | present in the audit log | Pass |

(Checks 15–16 run twice in the script, for two accounts, giving 31 results.)

### B. The real app against a real server (`integration_test/live_server_test.dart`, automated under Xvfb)

| ID | Case | Expected | Result |
|---|---|---|---|
| TC-30 | Sign up in the app, confirm the 8-digit code from the email | Account PIN screen, then the Account screen | Pass |
| TC-31 | Open the console as an admin without two-step | refused with instructions | Pass |
| TC-32 | Turn on two-step from the real dialog | "Two-step sign-in is on"; console opens | Pass |
| TC-33 | Import the custodian key | "Imported key" | Pass |
| TC-34 | Open a record with a reason | record readable, "Recorded by" shown | Pass |

### C. Widget-level flows (`flutter test`, automated, selected)

| ID | Case | Expected | Result |
|---|---|---|---|
| TC-35 | Demo session: check-ins answered, hold-to-send SOS, "I'm safe" | trays and halos update; SOS resolves | Pass |
| TC-36 | Invite code with a typo | precise inline message | Pass |
| TC-37 | Admin PIN: wrong PINs, lockout countdown, auto-lock | locked, countdown, relocks | Pass |
| TC-38 | Account PIN: lock, wrong PIN, unlock, forgot PIN signs out, change PIN, auto-lock keeps a draft | all as specified | Pass |
| TC-39 | Urgent safety report | sent as `critical` with diagnostics | Pass |
| TC-40 | Radar with Bluetooth-only peers | spaced evenly, no "N", caption "direction unknown" | Pass |
| TC-41 | Field limits and local checks | capped at `ProtocolLimits`; bad email or empty password refused with no request sent | Pass |

### D. Live hosted deployment (manual, 9–10 November 2026)

| ID | Case | Expected | Result |
|---|---|---|---|
| TC-42 | `GET /v1/health` from the internet | `ok: true, database: up` | Pass |
| TC-43 | Unauthenticated and forged-token requests to `/v1/me`, `/v1/admin/*` | 401 | Pass |
| TC-44 | CORS preflight from an unknown origin | no `Access-Control-Allow-Origin` | Pass |
| TC-45 | Plain HTTP | 301 to HTTPS | Pass |

| ID | Case | Expected | Result |
|---|---|---|---|
| TC-46 | Sign-up email through Brevo | code arrives in the inbox | Pass (after the `MAIL_FROM` and certificate fixes) |
| TC-47 | Daily backup to Supabase Storage | encrypted file in a private bucket | Pass (after the certificate, slot and URL fixes) |
| TC-48 | Supabase data API with the public key | no table readable or writable; RLS on all 14 tables | Pass (after the lock-down fix) |
| TC-49 | Database migrations on deploy | `schema_migrations` at the latest version | Pass |
| TC-50 | Supabase security advisor | no critical findings | Pass |

### F. Bluetooth LE between Android phones (manual, 13 November 2026)

| ID | Case | Expected | Result |
|---|---|---|---|
| TC-51 | Two phones, one creates a session, the other joins by QR | both appear in each other's tray within seconds | Pass |
| TC-52 | Join by typed code, including one with a typo | typo refused with a precise message; correct code joins | Pass |
| TC-53 | Walk apart and back | distance on the pinwheel rises and falls; peer card updates live | Pass |
| TC-54 | Distance against a known spacing | figure within a few metres | Pass with limitation: signal strength moves with bodies, walls and how the phone is held, so the figure is close, not exact; the app labels it "about" and jitters it by design |
| TC-55 | Cross the boundary circle | banner, haptic and notification on the other phone | Pass |
| TC-56 | Check in on one person, then everyone | delivered receipts; answers appear in the tray | Pass |
| TC-57 | Hold to send SOS; "I'm safe" | SOS reaches every phone with a halo and notification; resolves everywhere | Pass |
| TC-58 | Compare safety codes and mark verified | codes match; badge shown | Pass |
| TC-59 | Leave and rejoin from history | rejoin without retyping the code | Pass |
| TC-60 | Bluetooth off on one phone | status tray with a one-tap "Turn on" | Pass |

### E. Continuous integration (every push)

| Workflow | Jobs | Result on `main` |
|---|---|---|
| Lint | analyze all packages, format check | Pass |
| Test | app tests with coverage, Linux integration, server + protocol on PostgreSQL 16 | Pass, 13 November 2026, after the two session fixes in 5.3.3 (Linux integration best-effort on the runner's mDNS) |
| Build | Android, web, Linux, iOS (no codesign), macOS | Pass; **Windows fails** and is non-blocking |
| Web demo | build and publish to GitHub Pages | Pass; live at https://beekontek.github.io/EchoPin/ (the repository's Pages source is set to GitHub Actions once, by hand) |

### 5.4.3 Evaluation of the results

- **Functional coverage.** Every user-facing flow has at least one automated test at widget level, and every server endpoint is exercised by the server suite; the 31 system checks cover the whole account, record and break-glass lifecycle as real clients would drive it.
- **Security properties hold under test**: no account enumeration through answers or timing (TC-04, TC-08, TC-13), no lock-out of the legitimate owner (TC-12), two-step cannot be enabled with a stolen token (TC-15), the server alone cannot read a record (TC-25/26), every opening is audited and alerted (TC-27/29), and the audit chain verifies against an anchor outside the database (TC-28).
- **Defects found were real and are fixed**: the testing phase found four deployment defects that would have stopped real use (certificates, mail sender, backup slot, exposed data API) and three logic defects (tie proof, SOS count, dialog crash). All have regression tests or are re-checked by the system test.
- **Hardware.** Bluetooth LE between Android phones works as designed for every flow (part F). The one limitation is distance accuracy: a signal-strength estimate cannot be exact, which the design already assumes (the radar says "about", jitters the figure, and a boundary alert is a prompt to look up rather than a claim about metres). Direction is not measured at all and the radar says so.
- **Known gaps**, stated rather than hidden: iOS and macOS builds are compiled in CI but not run on devices; the Windows build fails in CI and no Windows host was available to diagnose it; the Linux mDNS integration test is reliable locally but not on GitHub's runners.

---

## 5.5 Installation

### 5.5.1 Requirements

| Target | Needs |
|---|---|
| Android phone | Android 7.0 (API 24) or newer; Bluetooth LE; about 80 MB free |
| iPhone | iOS 13 or newer; built and signed on a Mac (see `docs/SETUP.md`) |
| Linux desktop | GTK 3, `libsecret`, Avahi (Wi-Fi sessions) |
| Web browser | any modern browser; demo mode only |
| Server (optional) | Docker, or Dart 3.12 + PostgreSQL 16–18; or the free hosted recipe |

### 5.5.2 Install the app on Android

**By QR code (recommended for a group):**

```
flutter build apk --release \
  --dart-define=ECHOPIN_SERVER_URL=https://your-server \
  --dart-define=ECHOPIN_CUSTODIAN_KEY=<custodian public key>
scripts/share_apk.sh
```

The script serves the APK on your Wi-Fi and prints a QR code. On each phone: scan it with the camera, tap **Download**, open the file, allow installs from the browser if asked, tap **Install**. Press Ctrl+C when done; the script closes the firewall port it opened.

**By cable:**

```
adb install -r build/app/outputs/flutter-apk/app-release.apk
```

Omit the two `--dart-define`s for a fully local build with no accounts.

### 5.5.3 Install on Linux

```
flutter build linux --release
tar -C build/linux/x64/release -czf EchoPin-linux-x64.tar.gz --transform
's,^bundle,EchoPin,' bundle
```

On the target machine: extract and run `EchoPin/echo_pin`. Keep the `lib/` and `data/` folders beside the executable.

### 5.5.4 Web demo

Published automatically from `main` to https://beekontek.github.io/EchoPin/ by `.github/workflows/pages.yml`. To host elsewhere: `flutter build web --release --base-href /your/path/` and serve `build/web/`.

### 5.5.5 Install the server

**Free hosted (about an afternoon)**, step by step in `docs/DEPLOY_HOSTED.md`:

1. Create a Supabase project (database + storage), a Brevo account with a verified sender, and a Render account.
2. From `server/`: `dart run bin/echopin_admin.dart init-keys --print-env` and `init-custodian`; store the printed secrets in a password manager.
3. Push the repository and apply `render.yaml` as a Render Blueprint, pasting `DATABASE_URL`, `ECHOPIN_KEYS`, `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`, `BREVO_API_KEY`, `MAIL_FROM` and `CUSTODIAN_PUBLIC_KEYS`. The server validates every setting at start-up and names the one that is wrong.
4. Create the first admin: `dart run bin/echopin_admin.dart create-admin --email you@example.org --name "Your Name"` with `DATABASE_URL` and `ECHOPIN_KEYS` set; sign in on the app and turn on two-step.
5. Set the GitHub repository variable `ECHOPIN_SERVER_URL` so the keep-awake workflow pings the free instance every 10 minutes.

**Self-hosted**: `cd server && docker compose up` (PostgreSQL included), or `dart run bin/server.dart` with a `.env` from `.env.example`. Details, configuration reference and the CLI are in `docs/SERVER.md`.

### 5.5.6 First run checklist

1. `GET https://your-server/v1/health` answers `{"ok":true,"database":"up"}`.
2. Sign up in the app; the 8-digit code arrives by email.
3. Choose an account PIN; open **Account**.
4. Admin: turn on two-step, open the **Console**, check **Backups** the next day and press **Check chain** under **Audit**.
5. Responders: import the custodian key under Console → key icon.

### 5.5.7 Uninstall and data removal

- Phone: uninstalling removes the device key, PINs and history (the keystore entries go with the app). The account itself is removed by an admin in Console → People.
- Server: data retention is automatic (records 365 days, diagnostics 90, closed reports 730); the audit log is never deleted. Accounts that have opened records cannot be deleted by design (their openings must remain attributable); disable them instead.

---

## Appendix: verification commands

```
 flutter analyze
dart format --output=none --set-exit-if-changed lib test integration_test
flutter test
(cd packages/echopin_protocol && dart test)
(cd server && tool/test.sh)
(cd server && tool/live_check.sh)
xvfb-run -a flutter test integration_test/lan_transport_test.dart -d linux
flutter build apk --release && flutter build linux --release && flutter build web --release
```